

BrokerVerse

Out-of-the-box insurance broking platform

Solution Architecture · Document 10 of 11

Data Archival, Housekeeping and Restoration

Product	BrokerVerse
Document	10 Data Archival, Housekeeping and Restoration
Version	1.0
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse Solution Architecture: 10 Data Archival, Housekeeping and Restoration
Document ID	BV-SA-10
Version	1.0
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, docker-compose.yml, render.yaml, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

- 1 Introduction..... 4
 - 1.1 Purpose and scope..... 4
- 2 What grows..... 5
- 3 Retention policy..... 7
 - 3.1 Regulatory context..... 7
 - 3.2 Recommended retention periods..... 7
- 4 Housekeeping..... 9
 - 4.1 Existing jobs..... 9
 - 4.2 Recommended housekeeping jobs..... 9
- 5 Archival..... 10
 - 5.1 Approach..... 10
 - 5.2 Restoration from archive..... 11
- 6 Sample-data purge before go-live..... 12
- 7 Related documents..... 13
- 8 Glossary..... 14

1 Introduction

1.1 Purpose and scope

This document describes how BrokerVerse data grows, which housekeeping the system performs today, the recommended retention periods and housekeeping jobs, the approach to archiving old records out of the online database, how archived data is restored on request, and how the sample-data purge script is used before go-live.

Recommended / to be confirmed by the business and DevOps: Retention periods in this document are proposals. They must be confirmed by the compliance officer against the regulations that apply to the broker, in particular the record-keeping rules of the Bureau of Internal Revenue (BIR) and the Insurance Commission (IC), and the Data Privacy Act of 2012 (personal data kept no longer than necessary for the declared purposes). This document gives the regulatory context in general terms only; it is not legal advice.

2 What grows

Data	Growth driver	Current handling in the baseline
Business and financial records (leads to journals, 60+ tables)	Every quotation, policy, receipt, voucher, claim	Kept indefinitely; financial documents are cancelled or reversed, not deleted; draft quotations can be deleted; leads and quotations have <code>deleted_at</code>
<code>audit_log</code>	Every change, sign-in related event, job run started by a user, report generation	No purge; no date index
<code>login_history</code>	Every password, two-factor and refresh attempt	No purge
<code>job_runs</code>	About 1,735 rows a day: <code>renewal-queue</code> every minute (1,440), <code>email-outbox</code> every 5 minutes (288), 7 daily jobs	No purge; queried on every scheduled run
<code>notifications</code>	Reminders, approvals, renewal and collection notices	Deleted only when a user deletes them in the bell menu
<code>email_outbox</code>	Every e-mail (full HTML body kept)	No purge; sent and failed messages stay
<code>refresh_tokens</code>	Every sign-in and every refresh (rotation)	No purge of expired or revoked rows
<code>password_resets</code>	Forgot-password requests	No purge (codes expire after <code>security.reset_code_minutes</code>)
<code>password_history</code>	Password changes	Trimmed to the last <code>security.password_history_count</code> (5) per user by <code>lib/password.js</code>
<code>generated_reports</code> and report files	Scheduled daily reports (3 a day) and user-generated reports	Rows and files older than <code>reports.retention_days</code> (90) deleted by the <code>daily-reports</code> job (only when it runs on its schedule)
Uploaded files (<code>UPLOAD_DIR</code>)	Documents, photos, statements, bordereaux, exports	No lifecycle; deleted only through the delete endpoint (uploader, module writer or administrator)
<code>job_queue</code>	Renewal batch notices	Items stay after completion
Status housekeeping	Policies past expiry, quotations past validity, renewals past grace	Jobs <code>policy-expiry</code> , <code>quote-expiry</code> , <code>renewal-pipeline</code> change the status (no deletion)

On the local test database the largest tables are already log and operational tables: `job_runs`, `audit_log`, `login_history` and `refresh_tokens` (row counts in document 03), after only a few days of scheduler operation and test runs.

Gap: Apart from generated reports and password history, the system has no retention or purge job. `job_runs` grows by about 630,000 rows a year and is scanned by the scheduler without a suitable index; `email_outbox` keeps every e-mail body including personal data; `refresh_tokens`, `login_history`, `notifications` and `audit_log` grow without limit. Recommended housekeeping jobs are listed in chapter 4.

Note: The local upload folder of the development system holds 823 files (4.9 MB) while its `documents` table lists 24 (the folder is shared with test runs and resets). In production, files without a database record (orphans) can arise

from failed uploads (a key is reserved before the file is written) and from restores; a reconciliation job is recommended (chapter 4).

3 Retention policy

3.1 Regulatory context

- **BIR:** books of accounts and their supporting documents (official receipts, invoices, vouchers, journals, withholding tax certificates such as BIR Form 2307) must be preserved for the period set by the revenue regulations, currently up to ten years from the date of the last entry (Revenue Regulations No. 17-2013 as amended), and be available for examination.
- **Insurance Commission:** insurance brokers must keep complete records of the business transacted (policies placed, premiums received and remitted, commissions) available for examination, for the period required by the Insurance Code and IC circulars.
- **Data Privacy Act of 2012 (Republic Act 10173):** personal data of leads, clients and claimants may be kept only as long as necessary for the purpose or as required by law, then disposed of securely; access and retention must be documented.

3.2 Recommended retention periods

Class / data (Recommended / to be confirmed by compliance)	Tables	Online (PostgreSQL)	Total, incl. archive
Financial records	receivables, receipts and lines / applications, journal vouchers and lines, entry matches, disbursements, invoice lists, cheques, petty cash, commissions, remittances and items, debit notes and collections, direct-bill items, reinsurance recoveries and reconciliations, incentive calculations and results, policy payments	Current year + 2 closed years	10 years after the end of the financial year
Policy and claim records	clients, quotations (converted), policies, endorsements, claims with history, renewals, cessions, treaties, bordereaux, documents	Life of the policy / claim + 2 years	10 years after policy expiry or claim closure
Leads and quotations not converted	leads, quotes (not converted), renewal quotes, win-back campaigns	2 years	3 years, then anonymise or delete (privacy)
Audit trail	audit_log, claim_field_changes, claim_history, collection_actions, renewal_activities, renewal_notices, remittance_approvals	2 years	10 years (supports financial records)
Security logs	login_history, password_resets	1 year	2 years
Operational logs	job_runs	90 days	1 year (aggregated)
Temporary data	refresh_tokens (expired or revoked), email_outbox (sent), notifications (read), job_queue (done), generated_reports	30 to 90 days	Same (no archive)
Reference and configuration	settings, masters, product templates, roles, report catalogue	While in use	Versions kept via audit log
Uploaded files	EFS UPLOAD_DIR	As the owning record	As the owning record

4 Housekeeping

4.1 Existing jobs

Job	Schedule	Housekeeping effect
daily-reports	05:00 (cron, UTC)	Generates the daily reports and deletes generated reports and their files older than <code>reports.retention_days</code>
policy-expiry	00:15	Status <code>expired</code> for policies past expiry
quote-expiry	00:30	Status <code>expired</code> for quotations older than <code>limits.quote_validity_days</code>
renewal-pipeline	05:30	Lapses renewals past <code>renewals.grace_period_days</code>
Password change	on change	Keeps only the last N password hashes

4.2 Recommended housekeeping jobs

Recommended / to be confirmed by the business and DevOps: Add a housekeeping handler to `backend/src/jobs/handlers.js` and a row to `backend/src/db/seeds/jobs.json` (for example daily at 02:30 Manila time), with retention periods as new settings in a `retention` group so the business can change them. Delete in batches (for example 5,000 rows per statement) to keep transactions short; record the counts in the job output.

Task	Rule	New setting (proposed)
Job run history	Delete <code>job_runs</code> older than 90 days (keep failed runs 1 year)	<code>retention.job_runs_days = 90</code>
E-mail outbox	Delete <code>sent</code> rows older than 90 days; <code>failed</code> older than 1 year	<code>retention.email_outbox_days = 90</code>
Refresh tokens	Delete rows expired or revoked more than 30 days ago	<code>retention.refresh_tokens_days = 30</code>
Password reset codes	Delete used or expired codes older than 30 days	<code>retention.password_resets_days = 30</code>
Notifications	Delete read notifications older than 180 days, unread older than 1 year	<code>retention.notifications_days = 180</code>
Job queue	Delete completed items older than 30 days	<code>retention.job_queue_days = 30</code>
Sign-in history	Archive then delete <code>login_history</code> older than 1 year	<code>retention.login_history_days = 365</code>
Audit log	Archive then delete <code>audit_log</code> older than 2 years (after the archive is verified)	<code>retention.audit_log_days = 730</code>
Orphan files	Report (do not delete automatically) files in <code>UPLOAD_DIR</code> without a <code>documents / generated_reports</code> row, and rows without a file	none; weekly report

Supporting indexes (document 03): `job_runs` (`job_id`, `started_at`), `email_outbox` (`status`, `created_at`), `audit_log` (`at`), `refresh_tokens` (`expires_at`).

5 Archival

5.1 Approach

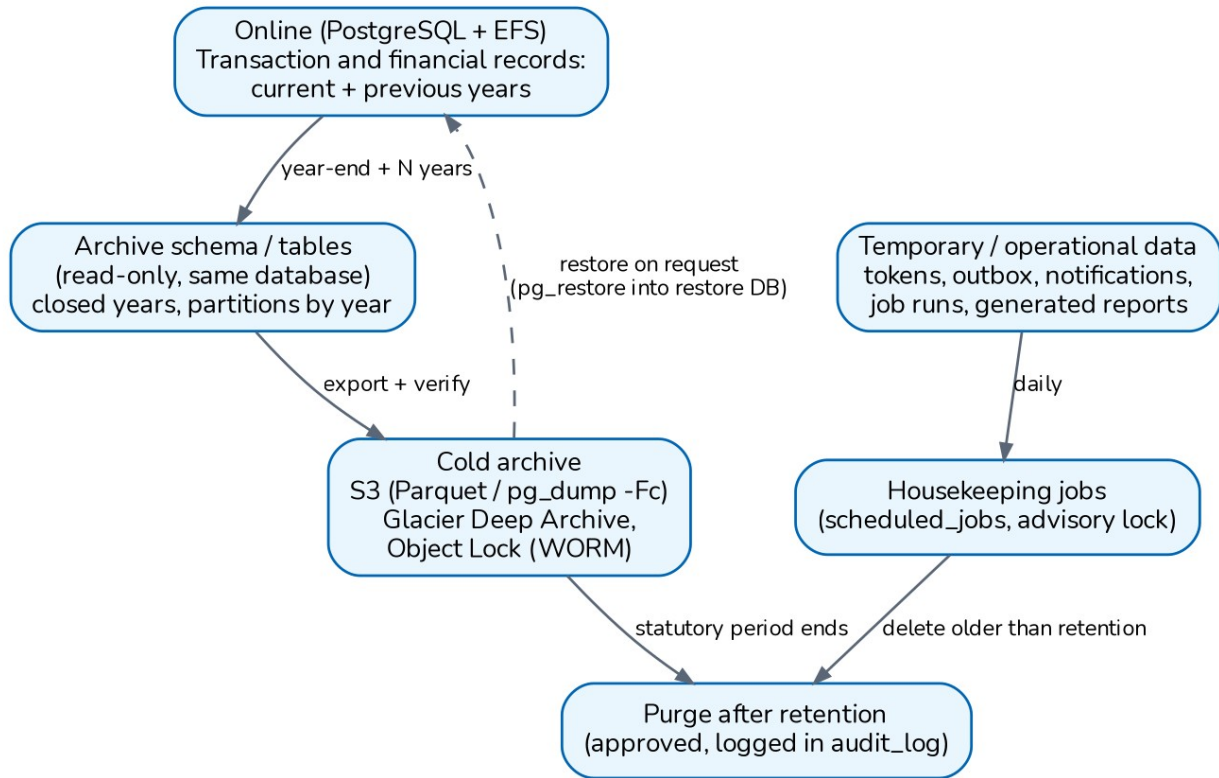


Figure 1. Data lifecycle: online, archive, purge (recommended)

Stage	Approach (Recommended / to be confirmed by the business and DevOps)
Online	Operational data stays in the main tables for the online period above; reports and screens work unchanged.
Archive in the database	For the high-volume log tables (<code>audit_log</code> , <code>login_history</code> , later <code>journal_lines</code>), convert to tables partitioned by year (declarative partitioning on <code>at / jv_date</code>) when the volume justifies it (for example above 10 million rows); old partitions can then be detached, exported and dropped in seconds without long deletes. Until then, move rows older than the online period into <code>archive.<table></code> tables with the same structure in a separate schema, in batches.
Cold archive	Once a year, after the financial year is closed and audited: export the closed year (per table, filtered by date or by the policy / financial year) with <code>COPY ... TO</code> as CSV or Parquet, or as a <code>pg_dump -Fc</code> of the archive schema, to an S3 bucket with Object Lock (compliance mode) and lifecycle to S3 Glacier Deep Archive; record a manifest (tables, row counts, checksums, date range, migration level <code>schema_migrations</code>). Files of the archived records are copied from EFS to the same bucket under their storage keys.
Purge	Only after the export is verified (row counts and checksums) and approved by the data owner: delete the archived rows from the online database; record the purge in <code>audit_log</code> . Financial records are purged only after the statutory period.
Referential integrity	Archive whole business units together (a policy with its quotation, bills, receipts, journals, commissions, claims and documents) or keep the referenced parent rows online; foreign keys (<code>NO ACTION</code>) prevent deleting a parent that is still referenced.

The yearly document-number counters (`sequences(name, period)`) stay online; old periods are small.

5.2 Restoration from archive

1. The data owner requests the records (for example a policy file of a given year for an audit or a claim dispute); the request is approved and logged.
2. DevOps restores the archive object from Glacier (standard retrieval up to 12 hours, bulk up to 48 hours for Deep Archive).
3. The data is loaded into a separate restore database (never into production): `pg_restore` of the archive dump, or `CREATE TABLE ... ; COPY ... FROM` of the CSV / Parquet files, at the migration level recorded in the manifest.
4. Read access is given to the requester through a read-only database login or a report extract; the files are provided from the restored S3 objects.
5. The restore database is deleted after the purpose is fulfilled; the access is recorded.

If records must be brought back into production (for example a reopened claim), load them with the same ids after a review of the dependent rows; ids are random text or serial values and do not collide with new records, and the document-number counters are not affected.

6 Sample-data purge before go-live

`backend/scripts/purge-sample-data.js` (`npm run purge:sample`) removes demo data from a database that was started with `SEED_SAMPLE_DATA` on (for example a UAT database promoted to production). It is a one-off go-live tool, not a housekeeping job.

Aspect	Behaviour
Safety	Refuses to run without <code>CONFIRM_PURGE=yes</code> ; dry run by default (row counts per table); <code>--execute</code> (alias <code>--no-dry-run</code>) deletes, in one transaction (any error rolls everything back). Stop the API instances first.
Removes	All rows of the transaction tables (leads to journals, claims, renewals, notifications, documents and generated-report records, job history, e-mail outbox, document-number counters), the sample master rows by natural key (fictional insurers, demo branches, signatories, referrers, reinsurers, incentive programmes, demo master records) and the sample users (<code>agent.*</code> , <code>fin.approver</code>). With <code>--keep-users=a,b</code> , every user except BrokerVerse and those listed.
Keeps	Reference data (settings, roles, permissions, scheduled jobs, masters, chart of accounts, product templates, report catalogue) and the audit trail (the purge is recorded in it); <code>--purge-audit</code> also empties <code>audit_log</code> and <code>login_history</code> .
Files	Uploaded files of removed records stay in <code>UPLOAD_DIR</code> ; clear the directory if nothing real was uploaded.
Afterwards	Set <code>SEED_SAMPLE_DATA=false</code> (or remove it) before starting the API, otherwise the sample data is seeded again.

Procedure: take a snapshot; stop the API; `CONFIRM_PURGE=yes npm run purge:sample` (review the counts); `CONFIRM_PURGE=yes npm run purge:sample -- --execute [--keep-users=...]`; set `SEED_SAMPLE_DATA=false`; start the API; check that Leads, Clients and Policies are empty.

Gap: `docs/GO_LIVE_CHECKLIST.md` section 7 shows `CONFIRM_PURGE=yes npm run purge:sample -- --apply` and a dry run without `CONFIRM_PURGE`; the script rejects `--apply` ("unknown option", it accepts `--execute` or `--no-dry-run`) and refuses any run, including the dry run, without `CONFIRM_PURGE=yes`. `docs/DEPLOY.md` shows the correct commands. Correct the checklist.

7 Related documents

This document is one of eleven that together form the BrokerVerse Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration (this document)	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

8 Glossary

Terms and abbreviations used in this document.

Term	Meaning
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
Audit log / audit trail	Table audit_log: who changed which record, when, from which IP, with the before and after values.
BIR	Bureau of Internal Revenue (Philippines): tax authority whose rules drive record keeping and withholding tax certificates.
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
IC	Insurance Commission (Philippines): regulator of insurers and brokers.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when SEED_SAMPLE_DATA is on.
UAT	User acceptance testing environment.